

Movie Ticket Booking System

TCS-504 — System Design | Assignment 1

SARTHAK NEGI

ML2, 54

B.Tech. CSE, Semester 5

A. Requirement Analysis

Functional Requirements

FR1 — Listing movies: the system shall display every Movie currently playing, showing its title, language, and duration in minutes.

FR2 — Listing shows: given a chosen movie, the system shall display only the Shows whose movie matches, showing each show's screen number and start time.

FR3 — Seat layout: given a chosen show, the system shall display every seat for that show grouped by seat type (SILVER / GOLD / PLATINUM), each marked [] if its ShowSeat status is AVAILABLE or [X] if BOOKED.

FR4 (given) — Booking: a customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.

FR5 — Pricing: the system shall price a booking by summing, for every selected seat, the fixed rate for its type — SILVER ₹150, GOLD ₹250, PLATINUM ₹400 — with no other charges.

FR6 (given) — Payment: exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.

FR7 — Ticket printing: once a booking's status is CONFIRMED, the system shall print a ticket containing the booking id, movie title, screen number, show start time, the list of booked seat numbers, and the total amount.

FR8 — Cancellation: given a valid booking id whose status is CONFIRMED, the system shall set the booking's status to CANCELLED and set every seat that booking held back to AVAILABLE. A booking that is not CONFIRMED shall not be cancellable.

Non-Functional Requirements

NFR1 — Modularity: one class per file; each class has exactly one responsibility, so a change to ticket formatting touches only TicketPrinter.

NFR2 — Extensibility: adding a new payment method (e.g. NetBanking) requires adding one new Payment subclass and one factory line, and must not require editing BookingService, Booking, or any existing Payment subclass.

NFR3 — Input validation / robustness: every console input (menu choice, seat number, booking id) is validated; invalid input produces a clear message and lets the program continue without crashing.

NFR4 — Consistency of seat state: a seat's state changes atomically as a side effect of a successful booking or a cancellation — never partially (e.g. a 2-seat booking where payment fails leaves BOTH seats AVAILABLE).

B. Noun–Verb Analysis

Noun found	Keep as a class?	Reason
Movie	Yes	has its own data and identity
Seat	Yes	has number, type, price
"seat layout"	No	a view of a Show's seats, not a thing → Show::printLayout()
Screen	Yes	has an identity (screen number) and owns seats
Cinema	Yes	the theatre itself; owns screens
Show	Yes	a movie+screen+time combination has its own identity and seat statuses
Customer	Yes	has name/phone, referenced by a booking
Booking	Yes	has an id, status, amount — a real business record
Payment	Yes	varies by method (UPI/Card/Cash) → abstraction + polymorphism
Ticket	No	never stored, only ever printed → TicketPrinter::printTicket()
"booking id"	No	an attribute of Booking, not a class of its own
"seat number"	No	an attribute (string) of Seat / ShowSeat
Price / amount	No	a double computed by PriceCalculator, not an entity
book / pay / cancel / list (verbs)	— (verb)	become methods: bookAndPay(), pay(), cancelBooking(), menu prints

• CLASSES

Core entity classes

Class	Knows (private data)	Does (public methods)	Must NOT do
Movie	title, language, durationMinutes	getters	know about shows/screens/pricing
Seat	seatNumber, type	getters; static type→price/string helpers	know if it's booked (that's ShowSeat)
Screen	screenNumber, vector<Seat>	addSeat, getSeats	know about shows or bookings
Cinema	name, vector<Screen>	addScreen, getScreens	run the menu or know about bookings
Show	movie*, screen*, startTime, vector<ShowSeat>	findSeat, printLayout	create bookings or take payments

Class	Knows (private data)	Does (public methods)	Must NOT do
ShowSeat	seatNumber, type, status	markBooked, markAvailable, isAvailable	calculate price or know which booking holds it
Customer	name, phone	getters	own/hold its bookings
Booking	id, show*, customer*, seats*, amount, status, payment*	confirm, fail, cancel	decide seat availability, run payment logic, print itself

Behaviour / service classes

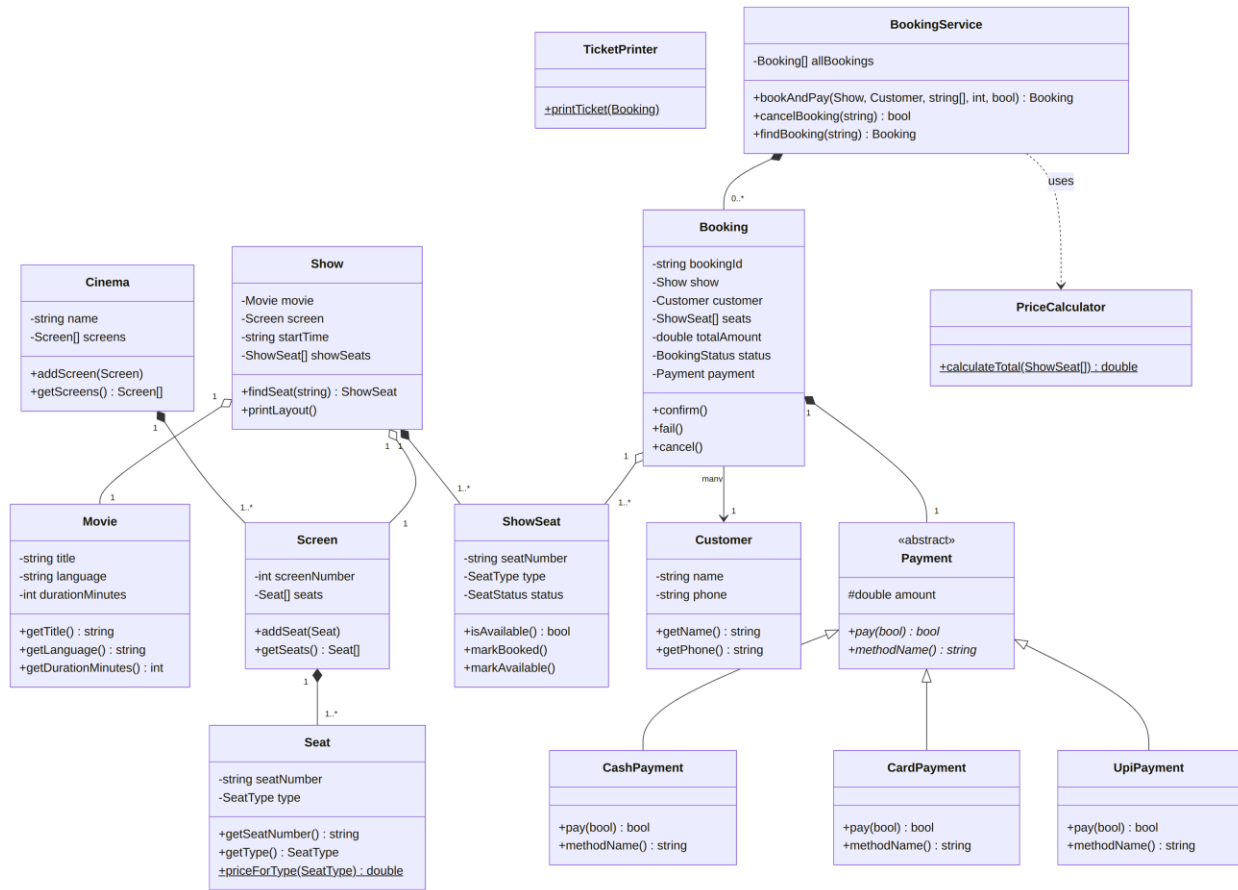
Class	Knows (private data)	Does (public methods)	Must NOT do
Payment (abstract)	amount	pay() pure virtual	know about Booking/Show/seats
Upi/Card/CashPayment	(inherited amount)	pay() override	anything outside "did this payment succeed"
PriceCalculator	nothing (stateless)	calculateTotal(seats)	touch seat status or create bookings
TicketPrinter	nothing (stateless)	printTicket(booking)	change booking/seat/payment state
BookingService	vector<Booking*>	bookAndPay, cancelBooking, findBooking	read console input or format a ticket
Cinema/main menu	sample data + service reference	the menu loop	contain business rules itself

C. Relationships (with the lifetime test)

Pair	Choice	Justification (lifetime test)
Cinema — Screen	Composition	A Screen has no meaning outside its Cinema; destroying the Cinema destroys its Screens (stored by value).
Screen — Seat	Composition	A physical seat only exists as part of a screen; destroy the Screen and its Seats go with it.
Show — Movie	Aggregation	(given) A Show borrows a Movie. Cancel the 6 PM show and "3 Idiots" still exists and still plays at 9 PM.
Show — Screen	Aggregation	A Show borrows a Screen (Screen*). Deleting one Show for Screen-1 does not delete Screen-1 — it is reused.
Show — ShowSeat	Composition	A ShowSeat has no meaning outside its Show; destroying the Show destroys its vector<ShowSeat>.

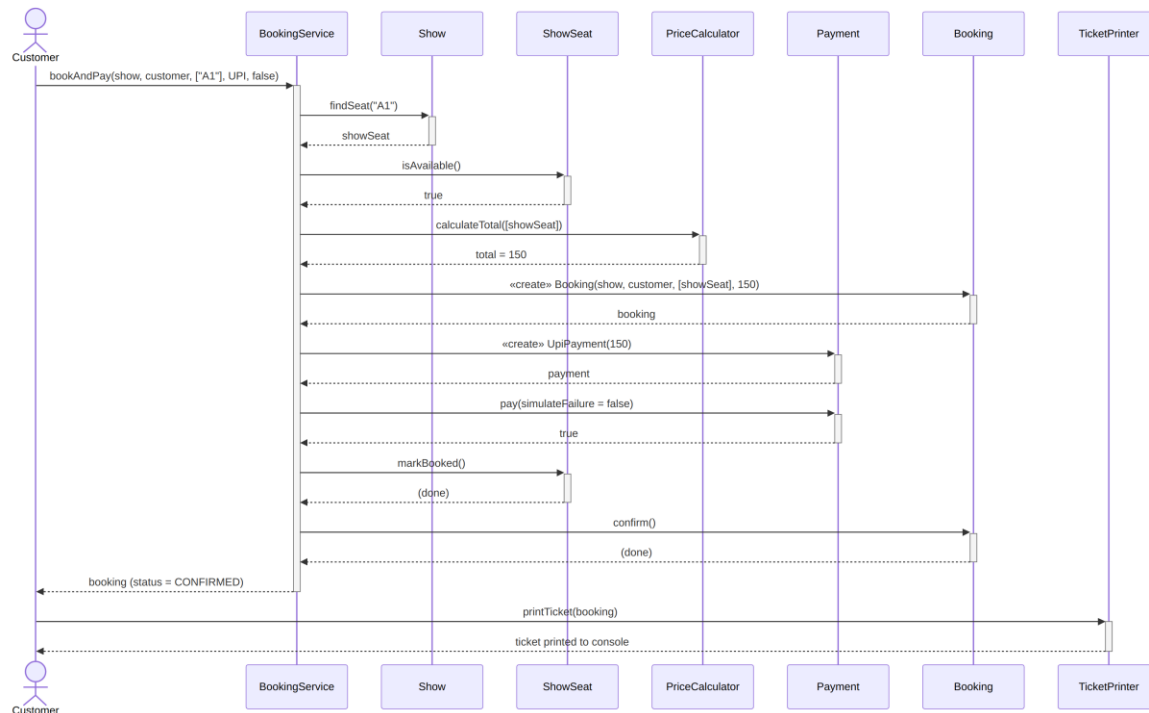
Pair	Choice	Justification (lifetime test)
Booking — Customer	Association	Neither owns the other; the Customer exists whether or not this Booking exists.
Booking — ShowSeat	Aggregation	Booking holds pointers to ShowSeats that live inside Show. Destroying the Booking does not destroy the seats.
Booking — Payment	Composition	A Payment record only makes sense as part of one Booking; Booking creates, owns, and deletes it.
Payment — UpiPayment	Inheritance	UpiPayment IS-A Payment (extends the abstract base) — a type relationship, not an ownership one.
BookingService — Booking	Composition	BookingService creates every Booking and is the only owner; destroying the service destroys all records.

D. Class Diagram



Notation: filled diamond (◆) = composition, hollow diamond (◇) = aggregation, open arrow = association, hollow triangle = inheritance, «abstract» marks the abstract Payment base.

E. Sequence Diagram — "customer books 1 seat and pays by UPI"



F. Modular Working Code + Demo Run

The full source (one class per file, no header files, per the course rule) is submitted alongside this document as 14 .cpp files: 01_Movie.cpp ... 13_BookingService.cpp, plus main.cpp. Build with:

```
g++ main.cpp -o ticketBooking
./ticketBooking      (or ticketBooking.exe on Windows)
```

Demo run 1 — happy path + ticket lookup (F1–F3, F5–F7)

```

===== MOVIE TICKET BOOKING =====
1. Movies  2. Book  3. Cancel  4. My tickets  0. Exit
Choose: 2
  [1] 3 Idiots   Hindi   170 min
  [2] Interstellar English 169 min
Choose movie: 1
  [1] Screen-1  06:00 PM
Choose show: 1
  SCREEN-1  06:00 PM | 3 Idiots
  SILVER   A1[ ] A2[ ] A3[ ] A4[ ]
  GOLD     B1[ ] B2[ ] B3[ ]
  PLATINUM C1[ ] C2[ ]
  ( [ ] = available  [X] = booked )
Seats (e.g. A1,B2): B2
Customer name: Sarthak
  
```

```

Customer phone: 7878912678
Pay by: 1.UPI 2.Card 3.Cash > 1
Simulate a FAILED payment for testing? (y/N): N
[UPI] Rs.250 paid successfully

===== TICKET =====
Booking ID : BK1001
Movie      : 3 Idiots
Screen     : Screen-1    06:00 PM
Seats      : B2
Amount     : Rs.250      Status: CONFIRMED
=====

--- Edge case: looking up the booking by id (Item F / Feature F7) ---
Choose: 4
Booking ID: BK1001
===== TICKET =====
Booking ID : BK1001
Movie      : 3 Idiots
Screen     : Screen-1    06:00 PM
Seats      : B2
Amount     : Rs.250      Status: CONFIRMED
=====

```

Demo run 2 — required edge cases (F4, F6, F8, invalid input)

```

--- Edge case 1: booking an already-BOOKED seat (Feature F4) ---
Choose: 2
Choose movie: 1
Choose show: 1
  SCREEN-1 06:00 PM | 3 Idiots
  SILVER   A1[ ] A2[ ] A3[ ] A4[ ]
  GOLD     B1[ ] B2[X] B3[ ]
  PLATINUM C1[ ] C2[ ]
  ( [ ] = available [X] = booked )
Seats (e.g. A1,B2): B2
Customer name: Aman
Customer phone: 456
Pay by: 1.UPI 2.Card 3.Cash > 2
Simulate a FAILED payment for testing? (y/N): N
  Seat B2 is already BOOKED. Booking cancelled, nothing changed.

--- Edge case 2: cancelling a CONFIRMED booking (Feature F8) ---
Choose: 3
Booking ID to cancel: BK1001
  Booking BK1001 cancelled. Seats are AVAILABLE again.

Choose: 4
Booking ID: BK1001
===== TICKET =====
Booking ID : BK1001
Movie      : 3 Idiots
Screen     : Screen-1    06:00 PM
Seats      : B2
Amount     : Rs.250      Status: CANCELLED
=====

```

```
--- Edge case 3: invalid menu / invalid booking id input, no crash ---  
Choose: 1  
Choose movie: 3  
    Invalid movie choice.  
Choose: 4  
Booking ID: 123  
    No such booking id.
```


G. SOLID Mapping

Single Responsibility Principle — Booking does NOT print the ticket (TicketPrinter does) and does NOT decide seat availability (BookingService does). A layout/formatting change only touches TicketPrinter; a pricing change only touches PriceCalculator.

Open/Closed Principle — Adding NetBanking needs only a new NetBankingPayment : public Payment class plus one line in the createPayment factory. BookingService::bookAndPay() calls payment->pay(...) through the abstract Payment interface and never changes when a new payment type is added.

Liskov Substitution Principle — Every Payment subclass (UpiPayment, CardPayment, CashPayment) can be used anywhere a Payment* is expected, with no extra setup call and no subclass behaving unexpectedly — BookingService calls pay() on the base pointer with no idea which concrete class it is.

Interface Segregation (applied as a judgment call) — Payment only declares pay() and methodName(); it deliberately does NOT declare refund(), since not every payment method in scope needs one and forcing every subclass to implement an unused refund() would violate ISP.

One thing deliberately NOT done

We did not build a fully pluggable payment registry (e.g. a map<string, function<Payment*(double)>> that new payment types register themselves into). Instead, createPayment() is a small factory function with an if-chain, so adding a new payment method still means touching one line in one factory function rather than being 100% closed to modification everywhere. This was an accepted, contained OCP compromise — building a self-registering plugin system was out of scope for an 8-feature console assignment, and BookingService itself, which is the requirement's actual concern, never needs to change.